

<https://www.halvorsen.blog>



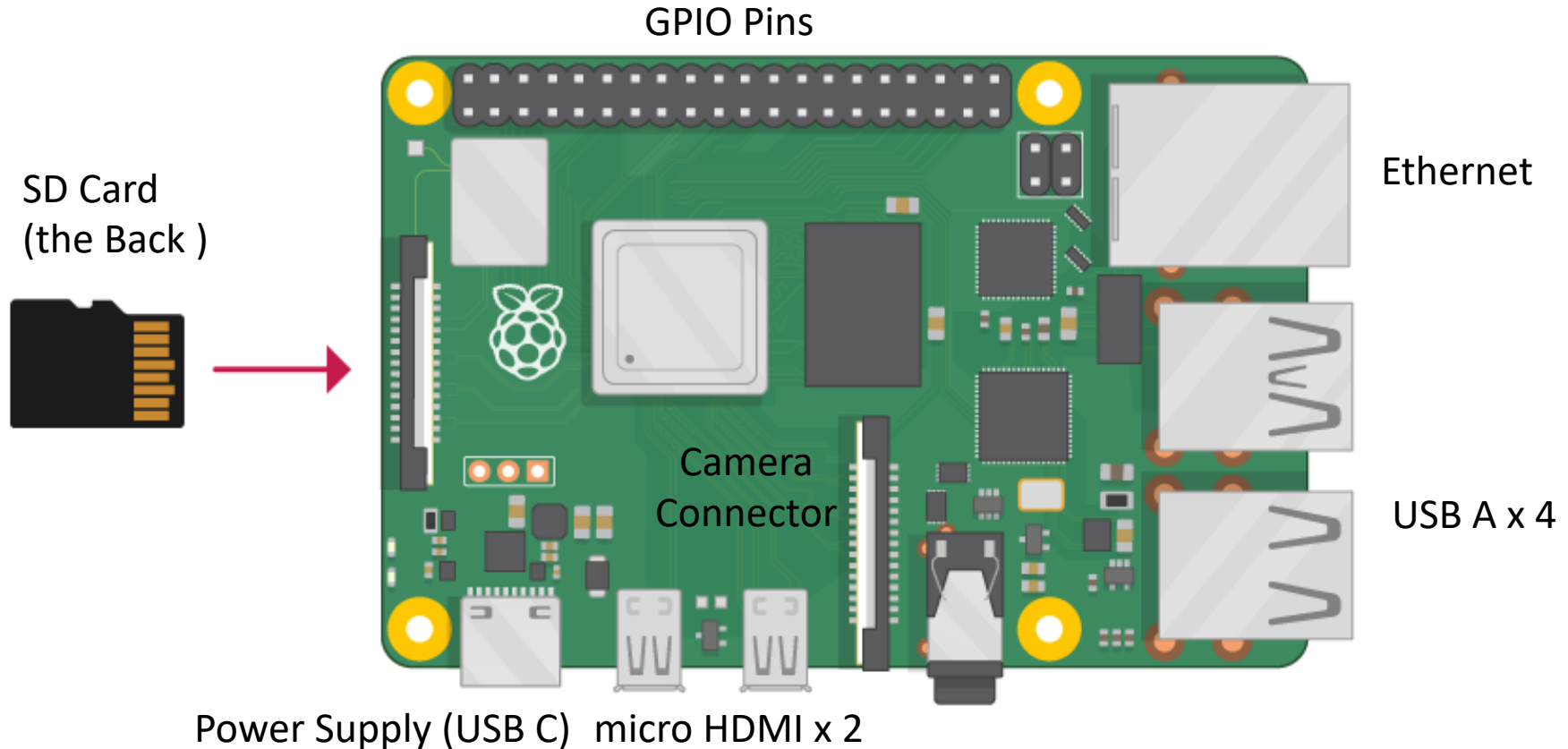
Raspberry Pi GPIO with Python

Hans-Petter Halvorsen

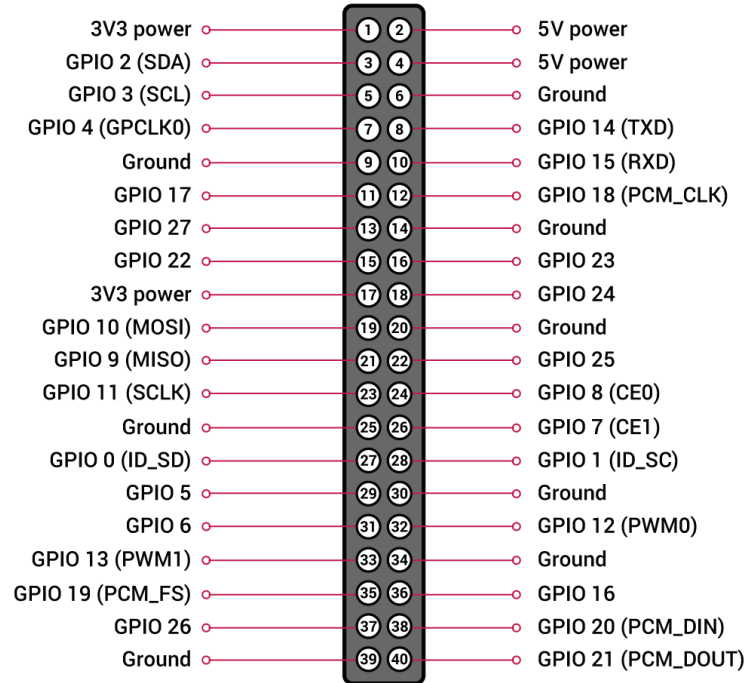
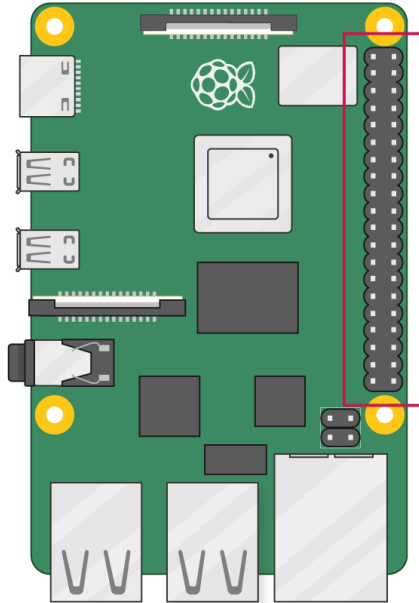
Contents

- Overview of GPIO
- LED
- PWM
- Push Button/Switch
- ADC (Analog to Digital Converter)
- TMP36
- ThingSpeak (Save Data to a Cloud Service)

Raspberry Pi



GPIO



A powerful feature of the Raspberry Pi is the GPIO (general-purpose input/output) pins. The Raspberry Pi has a 40-pin GPIO header as seen in the image

GPIO Features

The GPIO pins are Digital Pins which are either True (+3.3V) or False (0V). These can be used to turn on/off LEDs, etc.

The Digital Pins can be either Output or Input.

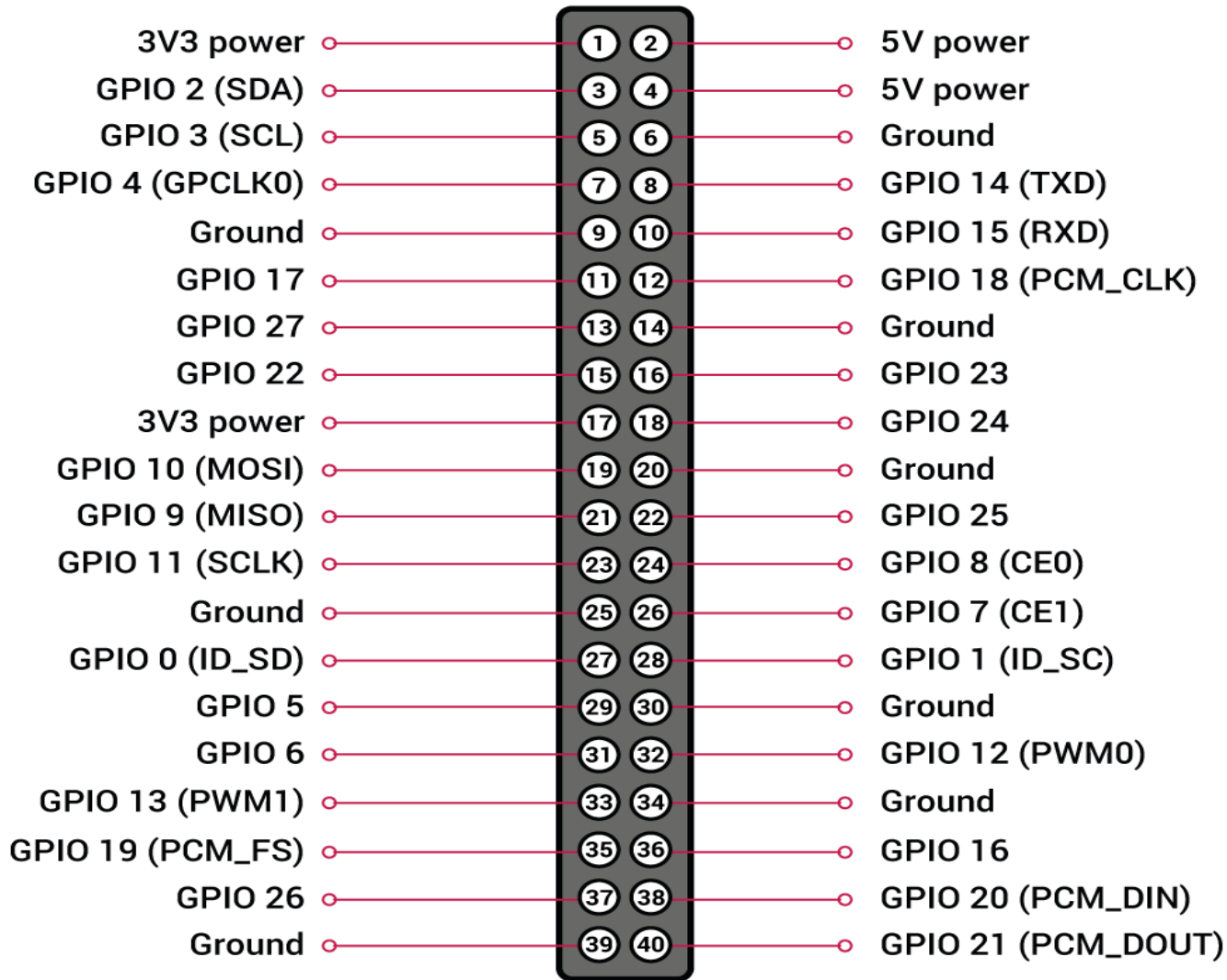
In addition, some of the pins also offer some other Features:

- PWM (Pulse Width Modulation)

Digital Buses (for reading data from Sensors, etc.):

- SPI
- I2C

GPIO



GPIO Zero

- The **GPIO Zero Python Library** can be used to communicate with GPIO Pins
- The **GPIO Zero Python Library** comes preinstalled with the Raspberry Pi OS (so no additional installation is necessary)

Resources:

- <https://www.raspberrypi.org/documentation/usage/gpio/python/>
- <https://pypi.org/project/gpiozero/>
- <https://gpiozero.readthedocs.io/en/stable/>
- <https://gpiozero.readthedocs.io/en/stable/recipes.html>

RPi.GPIO

- Rpi.GPIO is a module controlling the GPIO pins on the Raspberry Pi
- RPi.GPIO is a more “low-level” Python Library than GPIO Zero. Actually, GPIO Zero is using RPi.GPIO
- The RPi.GPIO Python Library comes preinstalled with the Raspberry Pi OS (so no additional installation is necessary)

<https://pypi.org/project/RPi.GPIO/>

<https://www.halvorsen.blog>

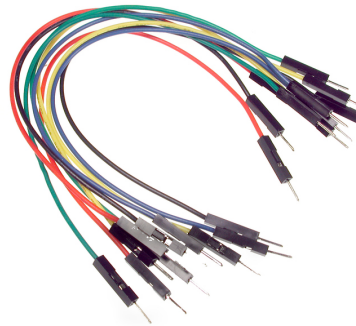


LED

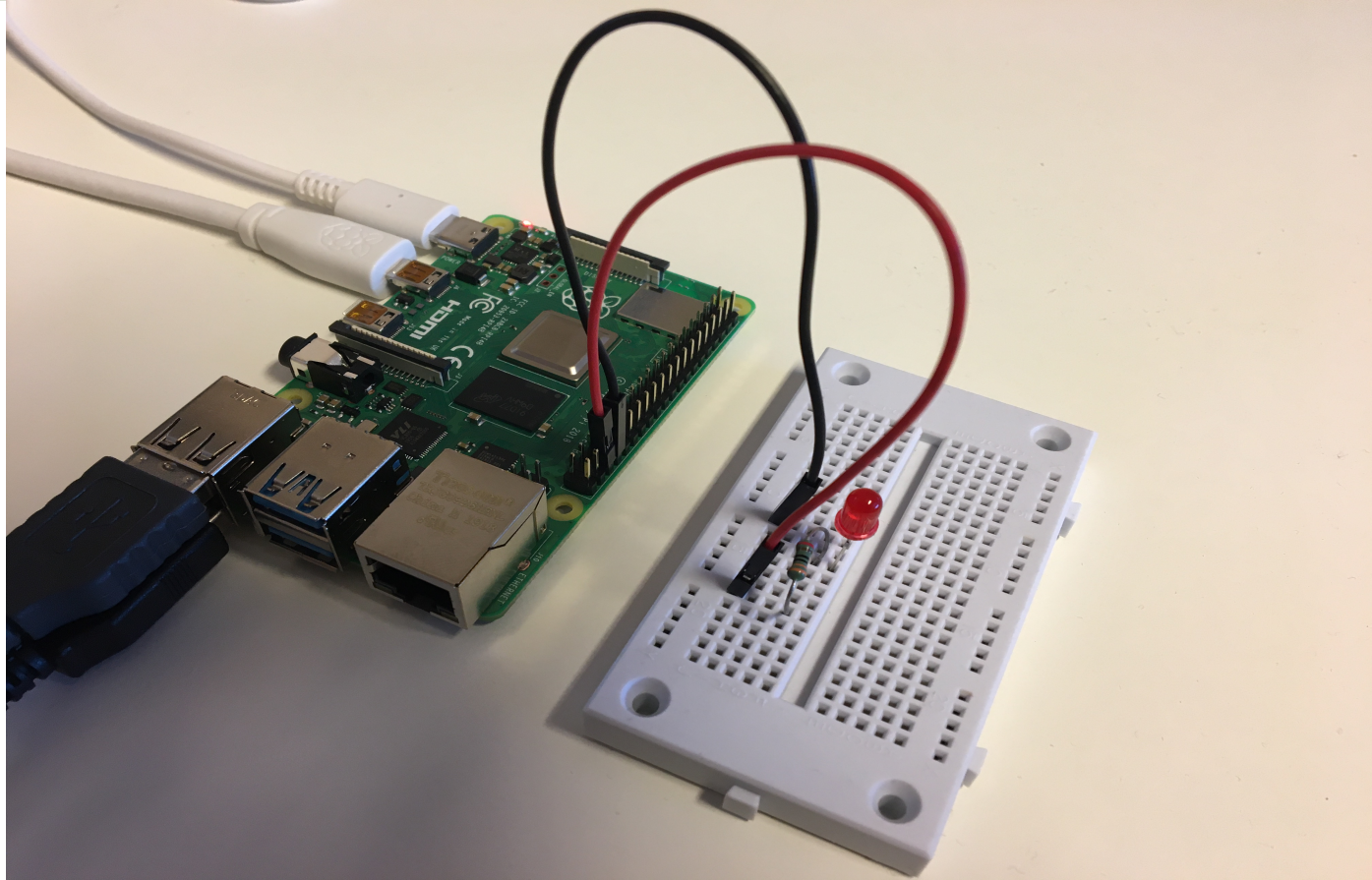
Hans-Petter Halvorsen

Necessary Equipment

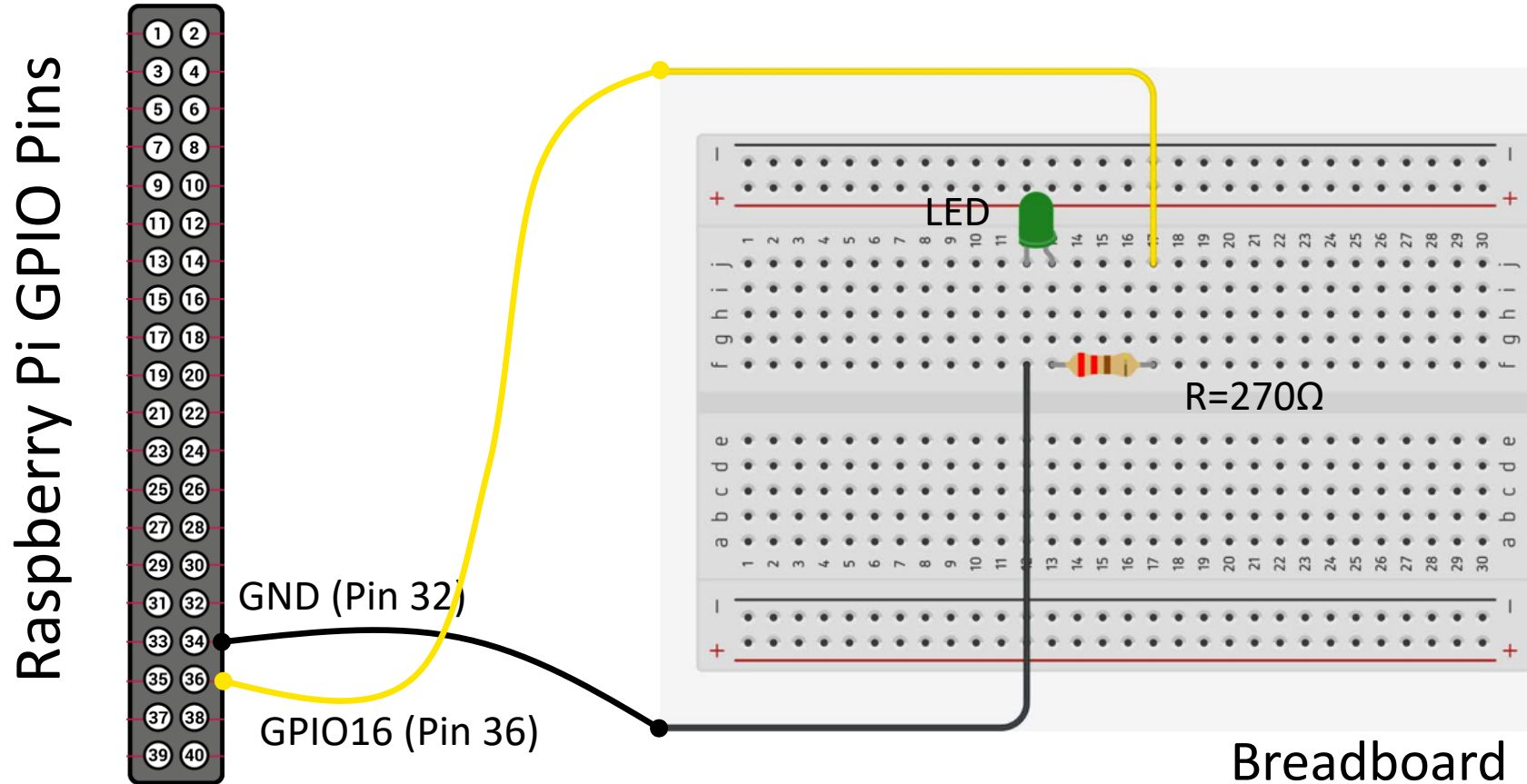
- Raspberry Pi
- Breadboard
- LED
- Resistor, $R = 270\Omega$
- Wires (Jumper Wires)



Setup and Wiring

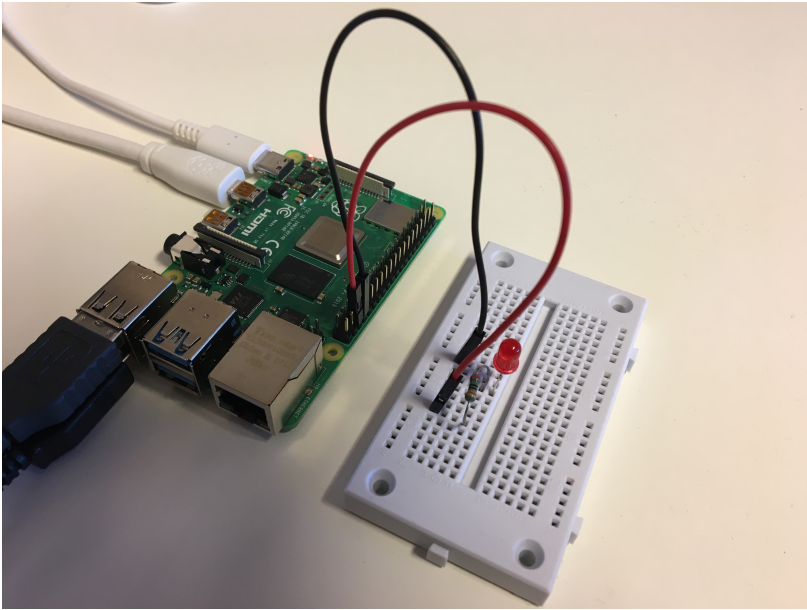


LED Example



LED Example

This Example “Runs for ever”

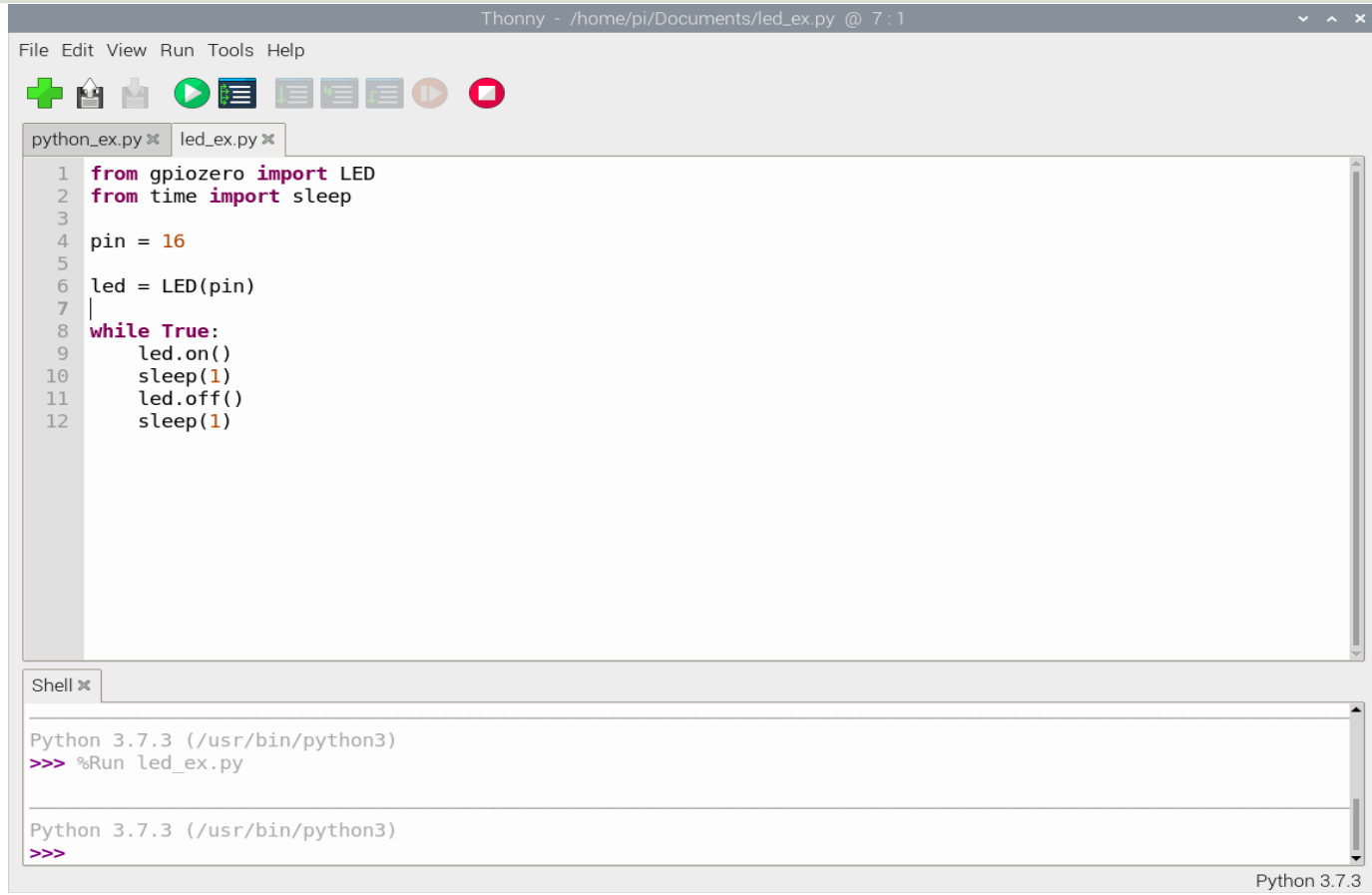


```
from gpiozero import LED
from time import sleep
```

```
pin = 16
led = LED(pin)
```

```
while True:
    led.on()
    sleep(1)
    led.off()
    sleep(1)
```

LED Example



The image shows a screenshot of the Thonny Python IDE. The window title is "Thonny - /home/pi/Documents/led_ex.py @ 7:1". The menu bar includes "File", "Edit", "View", "Run", "Tools", and "Help". The toolbar contains icons for opening a file, saving, running, and other functions. Two tabs are open: "python_ex.py" and "led_ex.py". The "led_ex.py" tab is active, showing the following Python code:

```
1 from gpiozero import LED
2 from time import sleep
3
4 pin = 16
5
6 led = LED(pin)
7
8 while True:
9     led.on()
10    sleep(1)
11    led.off()
12    sleep(1)
```

Below the code editor is a "Shell" window. It shows the command prompt "Python 3.7.3 (/usr/bin/python3)" and the command ">>> %Run led_ex.py". The shell is currently waiting for input, indicated by ">>>".

```
Python 3.7.3 (/usr/bin/python3)
>>> %Run led_ex.py

Python 3.7.3 (/usr/bin/python3)
>>>
```

Python 3.7.3

LED Example

This example turns a LED on/off 10 times

```
from gpiozero import LED
from time import sleep

pin = 16
led = LED(pin)

N = 10
for x in range(N):
    led.on()
    sleep(1)
    led.off()
    sleep(1)
```



PWM

Pulse Width Modulation

Hans-Petter Halvorsen

PWM

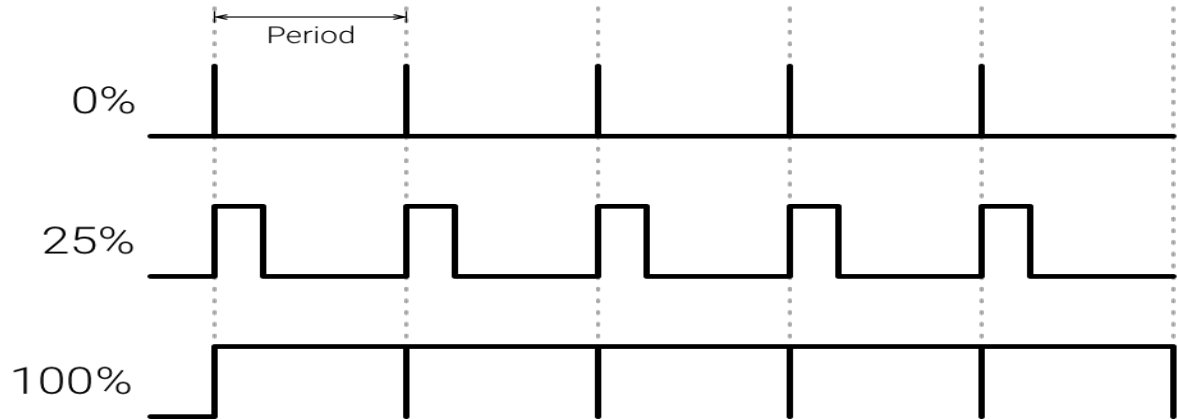
PWM is a digital (i.e., square wave) signal that oscillates according to a given *frequency* and *duty cycle*.

The frequency (expressed in Hz) describes how often the output pulse repeats.

The period is the time each cycle takes and is the inverse of frequency.

The duty cycle (expressed as a percentage) describes the width of the pulse within that frequency window.

You can adjust the duty cycle to increase or decrease the average "on" time of the signal. The following diagram shows pulse trains at 0%, 25%, and 100% duty:

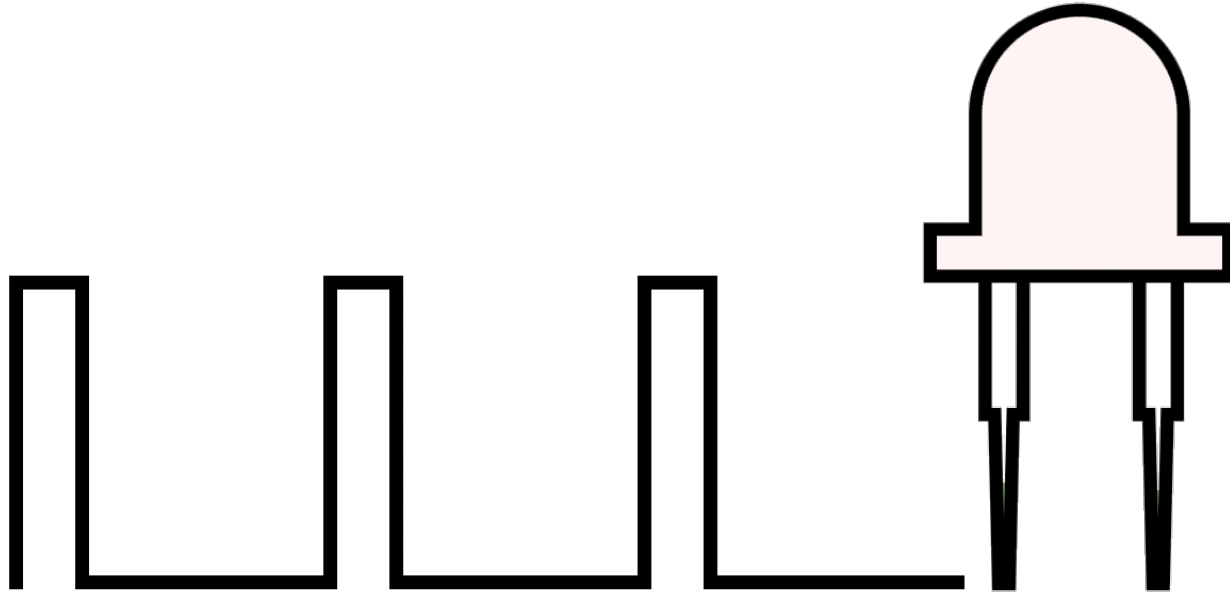


Controlling LED Brightness using PWM

- We've seen how to turn an LED on and off, but how do we control its brightness levels?
- An LED's brightness is determined by controlling the amount of current flowing through it, but that requires a lot more hardware components.
- A simple trick we can do is to flash the LED faster than the eye can see!
- By controlling the amount of time the LED is on versus off, we can change its perceived brightness.
- This is known as *Pulse Width Modulation* (PWM).

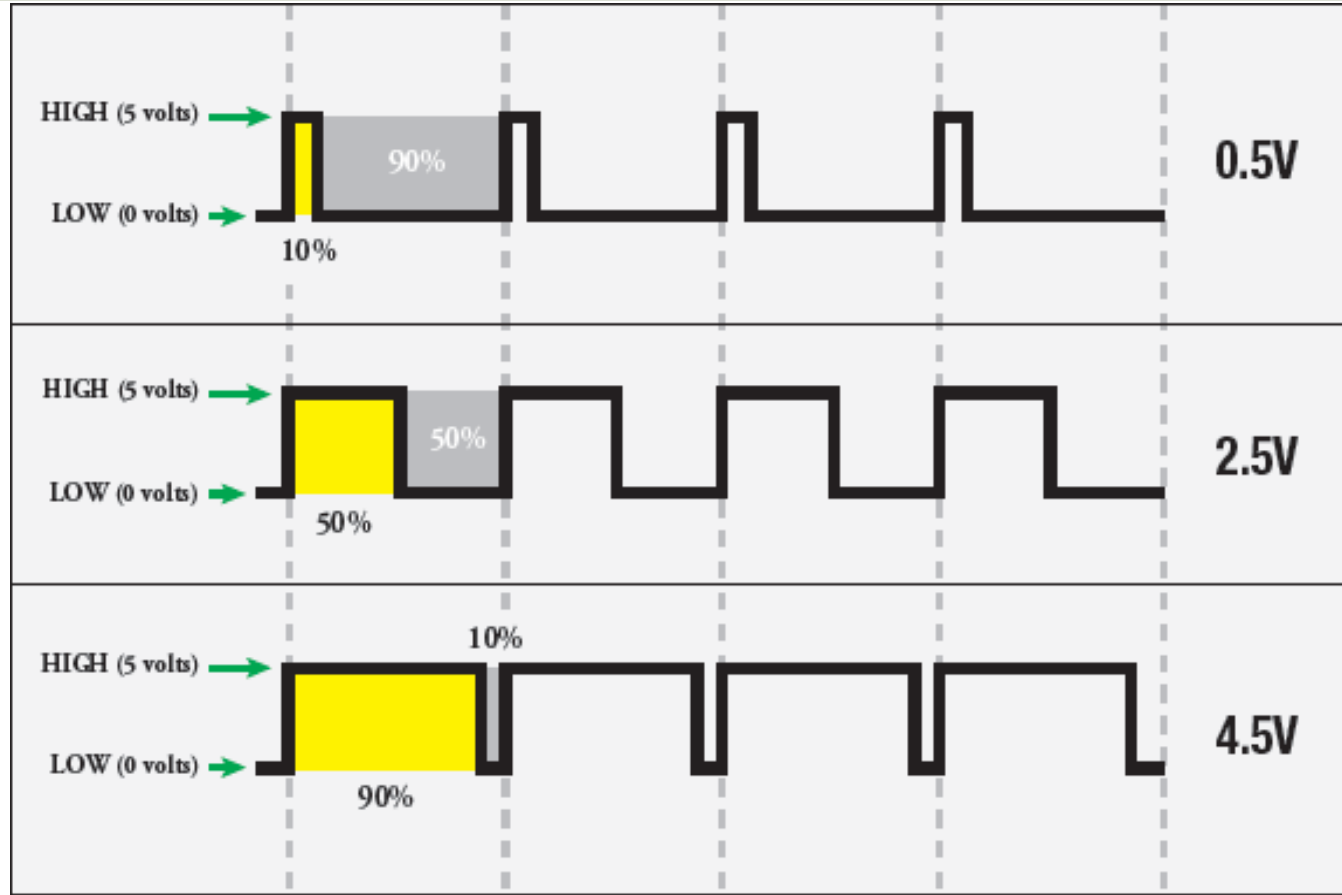
Controlling LED Brightness using PWM

Below we see how we can use PWM to control the brightness of a LED



PWM as “Analog Out”

The Raspberry Pi has no real Analog Out pins, but we can use a PWM pin. PWM can be used to control brightness of a LED, control the speed of a Fan, control a DC Motor, etc.



GPIO Zero

PWM

```
from time import sleep
import numpy as np
from gpiozero import PWMLED

pin = 23
led = PWMLED(pin)

start = 0
stop = 1
step = 0.1
level = np.arange(start, stop, step)

for x in level:
    led.value = x
    sleep(1)

led.off()
```

RPi.GPIO

PWM

```
import time
import RPi.GPIO as GPIO

# Pin definitions
led_pin = 23

# Use "GPIO" pin numbering
GPIO.setmode(GPIO.BCM)

# Set LED pin as output
GPIO.setup(led_pin, GPIO.OUT)

# Initialize pwm object with 50 Hz and 0% duty cycle
pwm = GPIO.PWM(led_pin, 50)
pwm.start(0)

pwm.ChangeDutyCycle(10)
time.sleep(2)
pwm.ChangeDutyCycle(50)
time.sleep(2)
pwm.ChangeDutyCycle(90)
time.sleep(2)

# Stop, cleanup, and exit
pwm.stop()
GPIO.cleanup()
```

<https://www.halvorsen.blog>

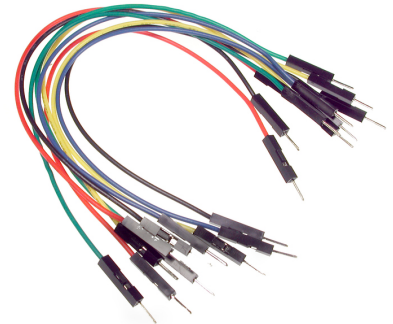
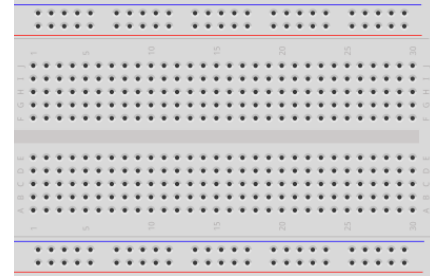
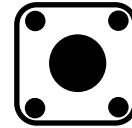
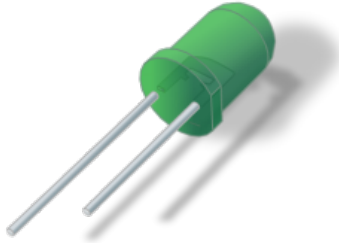


Push Button

Hans-Petter Halvorsen

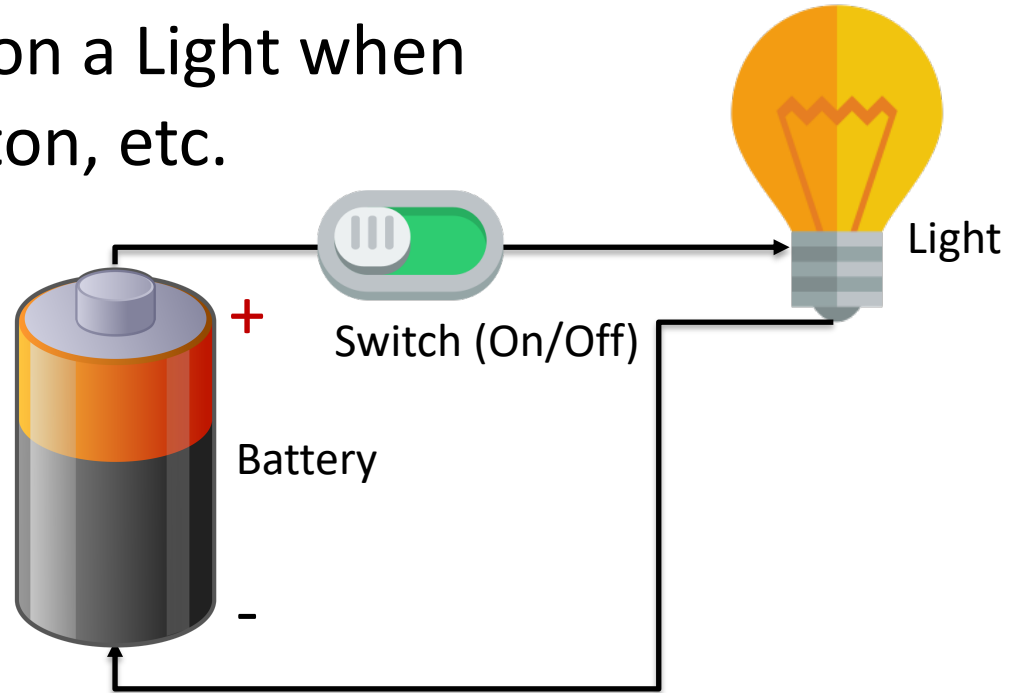
Necessary Equipment

- Raspberry Pi
- Breadboard
- Push Button
- LED
- Resistors, $R = 270\Omega$, $R = 10k\Omega$
- Wires (Jumper Wires)

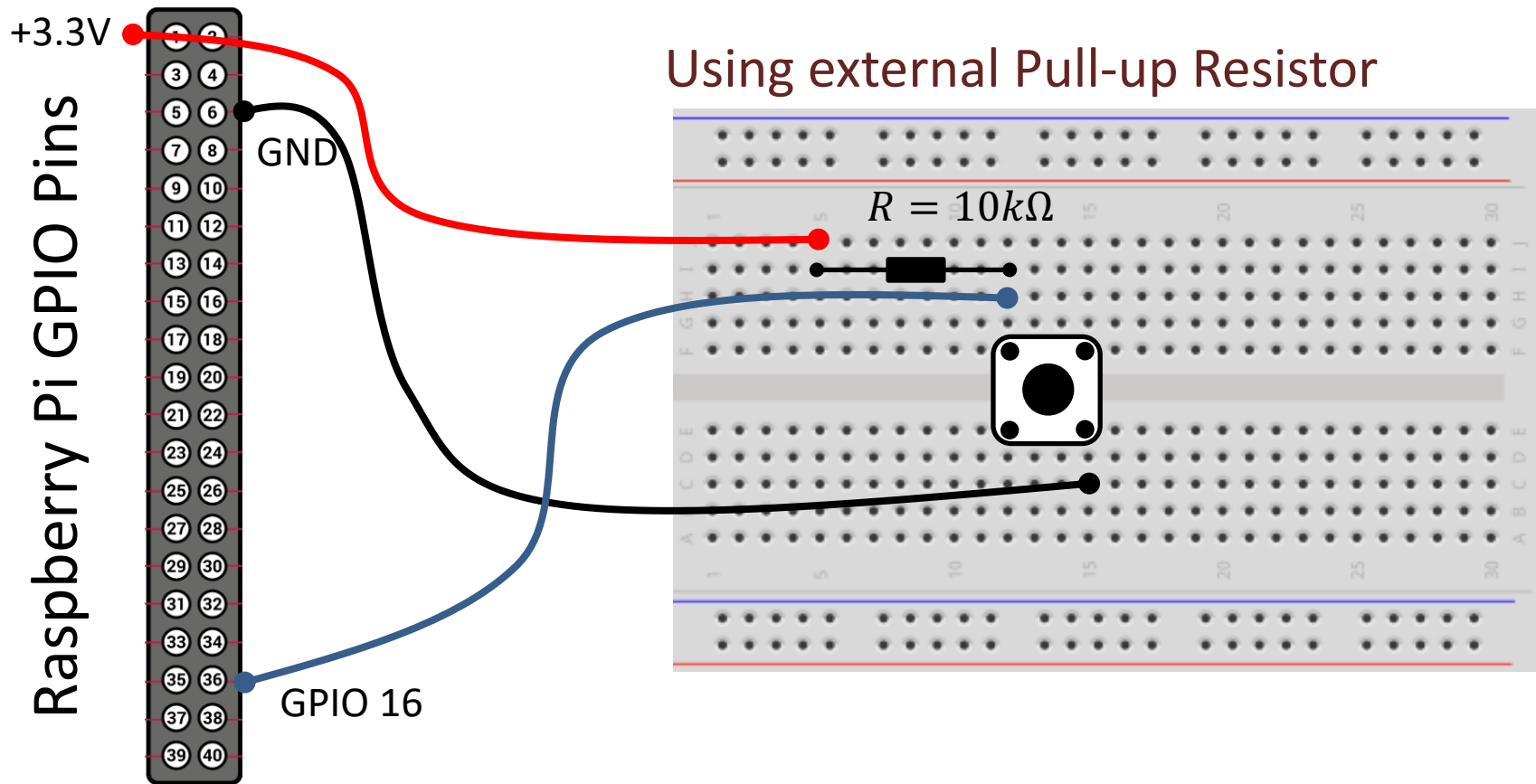


Push Button/Switch

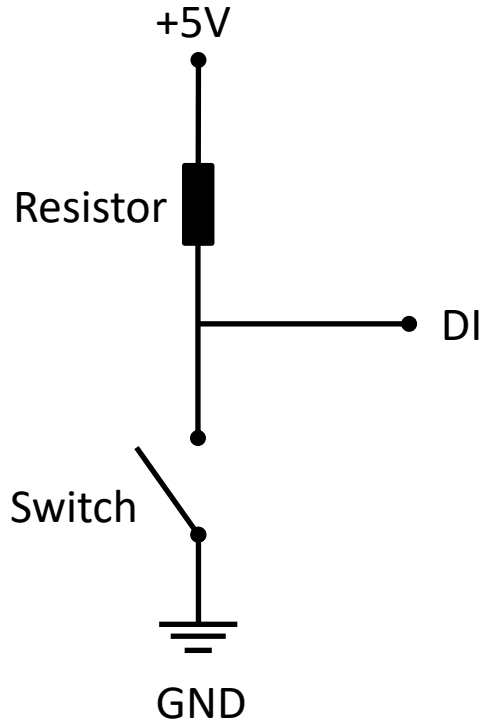
- Pushbuttons or switches connect two points in a circuit when you press them.
- You can use it to turn on a Light when holding down the button, etc.



Button Setup



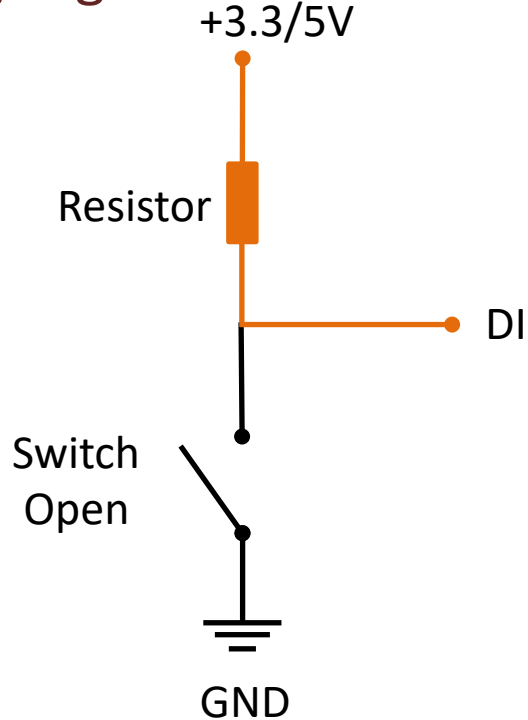
Pull-up Resistor



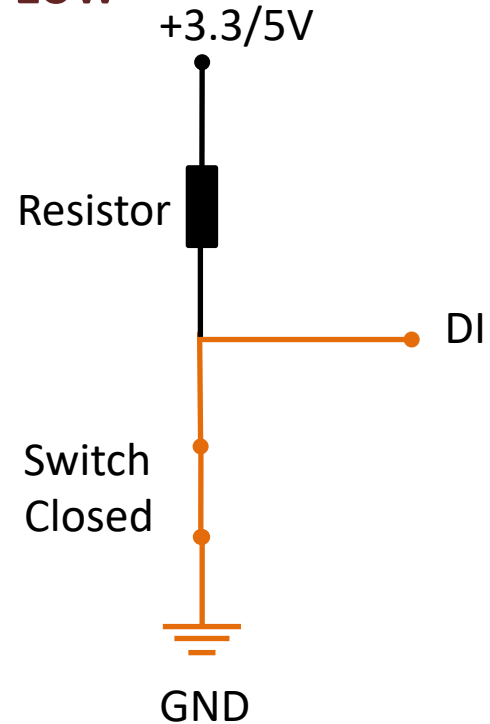
- When the pushbutton is open (unpressed) there is a connection between 3.3/5V and the DI pin.
- This means the default state is **True** (High).
- When the button is closed (pressed), the state goes to **False** (Low).

Pull-up Resistor

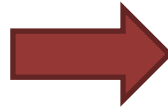
True/High



False/Low



We Push the Button

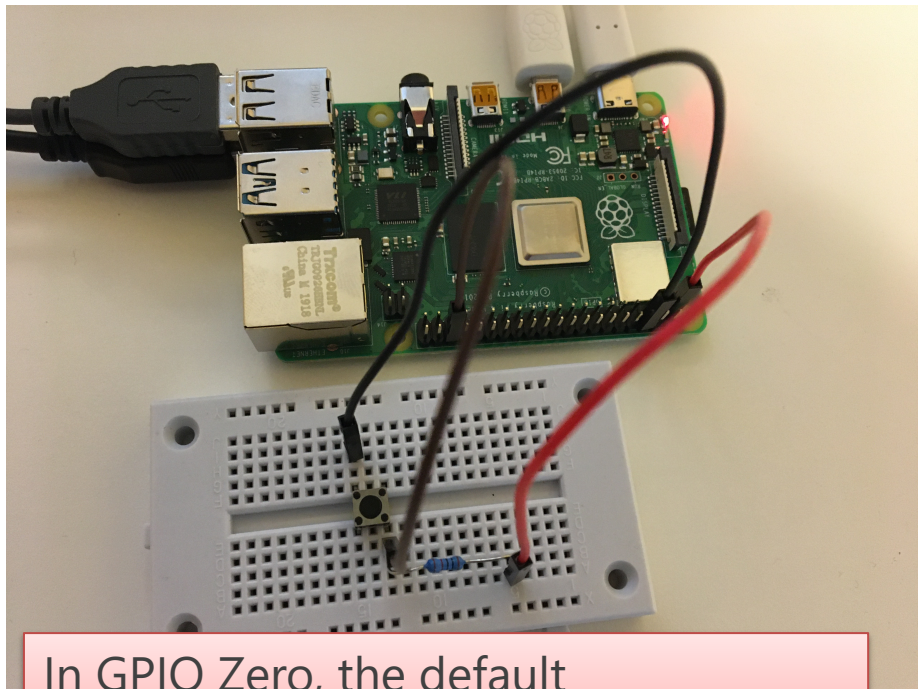


Pull-down/Pull-up Resistor

Why do we need a pull-up or pull-down resistor in the circuit?

- If you disconnect the digital I/O pin from everything, it will behave in an irregular way.
- This is because the input is "floating" - that is, it will randomly return either HIGH or LOW.
- That's why you need a pull-up or pull-down resistor in the circuit.

Button Example



In GPIO Zero, the default configuration for a button is pull-up

```
from gpiozero import Button
from time import sleep
pin = 16
button = Button(pin)

while True:
    if button.is_pressed:
        print("Pressed")
    else:
        print("Released")
    sleep(1)
```

Button Example

Thonny - /home/pi/Documents/button_ex.py @ 9:32

File Edit View Run Tools Help

button_ex.py button_ex2.py button_ex3.py

```
1 from gpiozero import LED, Button
2 from time import sleep
3
4 pin = 16
5 button = Button(pin)
6
7 while True:
8     if button.is_pressed:
9         print("Button Pressed")
10    else:
11        print("Button Released")
12    sleep(1)
```

Assistant

The code in [button_ex3.py](#) looks good.

If it is not working as it should, then consider using some general [debugging techniques](#).

[Was it helpful or confusing?](#)

In GPIO Zero, the default configuration for a button is pull-up

We have wired according to pull-up.
This means:
Button Pressed -> True
Button Not Pressed -> False

Shell

```
Button Released
Button Released
Button Released
Button Pressed
Button Released
Button Pressed
Button Released
Button Pressed
```

Python 3.7.3

Button Ex.2

Here is the **RPi.GPIO** Python Library used

In RPi.GPIO, the default configuration for a button is pull-down

We have wired according to pull-up.
This means:

Button Pressed -> False

Button Not Pressed -> True

```
import time
import RPi.GPIO as GPIO

# Pins definitions
btn_pin = 16

# Set up pins
GPIO.setmode(GPIO.BCM)
GPIO.setup(btn_pin, GPIO.IN)

# If button is pushed, light up LED
try:
    while True:
        if GPIO.input(btn_pin):
            print("Button Released")
        else:
            print("Button Pressed")
            time.sleep(1)

# When you press ctrl+c, this will be called
finally:
    GPIO.cleanup()
```

File Edit View Run Tools Help



button_ex.py x button_ex2.py x button_ex3.py x

Assistant x

```
1 import time
2 import RPi.GPIO as GPIO
3
4 # Pins definitions
5 btn_pin = 16
6
7 # Set up pins
8 GPIO.setmode(GPIO.BCM)
9 GPIO.setup(btn_pin, GPIO.IN)
10
11 # If button is pushed, light up LED
12 try:
13     while True:
14         if GPIO.input(btn_pin):
15             print("Button Released")
16         else:
17             print("Button Pressed")
18             time.sleep(1)
19
20 # When you press ctrl+c, this will be called
21 finally:
22     GPIO.cleanup()
```

Shell x

```
Button Released
Button Released
Button Released
Button Released
Button Released
Button Released
Button Released
Button Pressed
Button Pressed
```

Button Ex.3

```
import time
import RPi.GPIO as GPIO

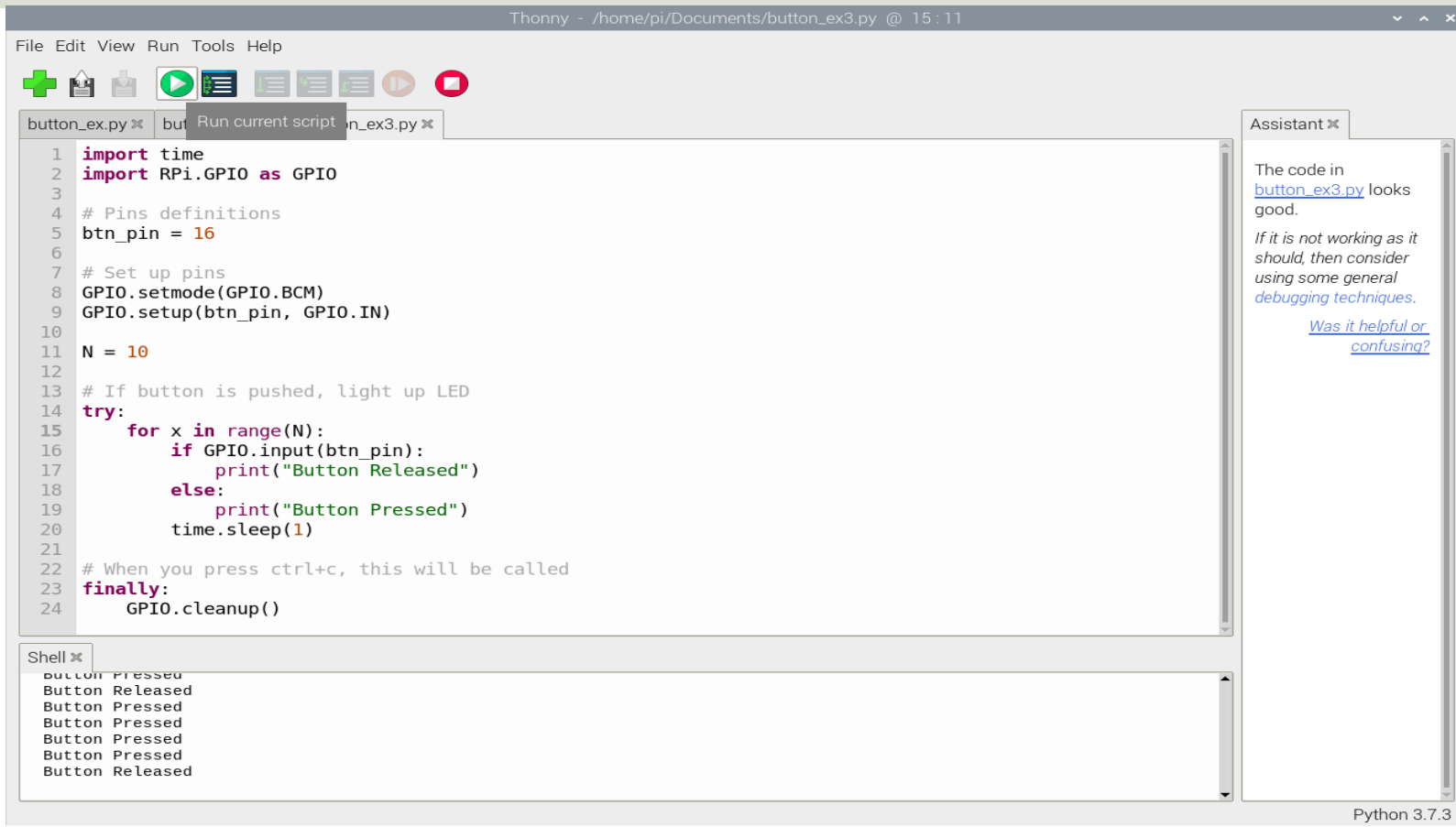
# Pins definitions
btn_pin = 16

# Set up pins
GPIO.setmode(GPIO.BCM)
GPIO.setup(btn_pin, GPIO.IN)

N = 10
# If button is pushed, light up LED
try:
    for x in range(N):
        if GPIO.input(btn_pin):
            print("Button Released")
        else:
            print("Button Pressed")
            time.sleep(1)

# When you press ctrl+c, this will be
called
finally:
    GPIO.cleanup()
```

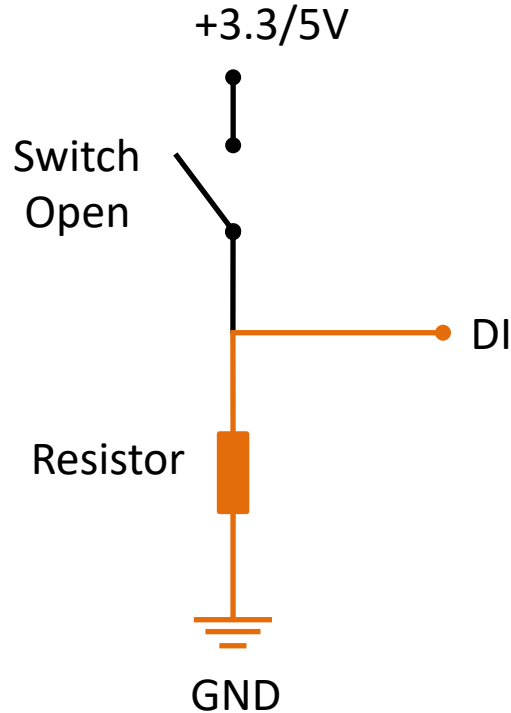
Button Example3



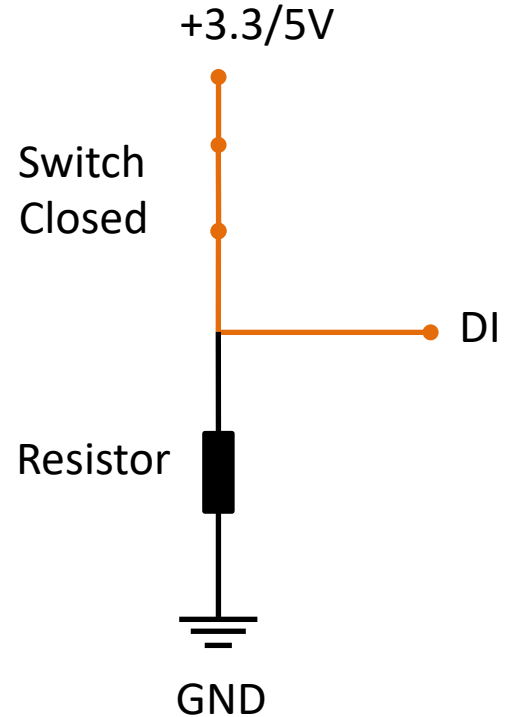
Pull-down Resistor

We could also have wired according to a “Pull-down” Resistor

False/Low

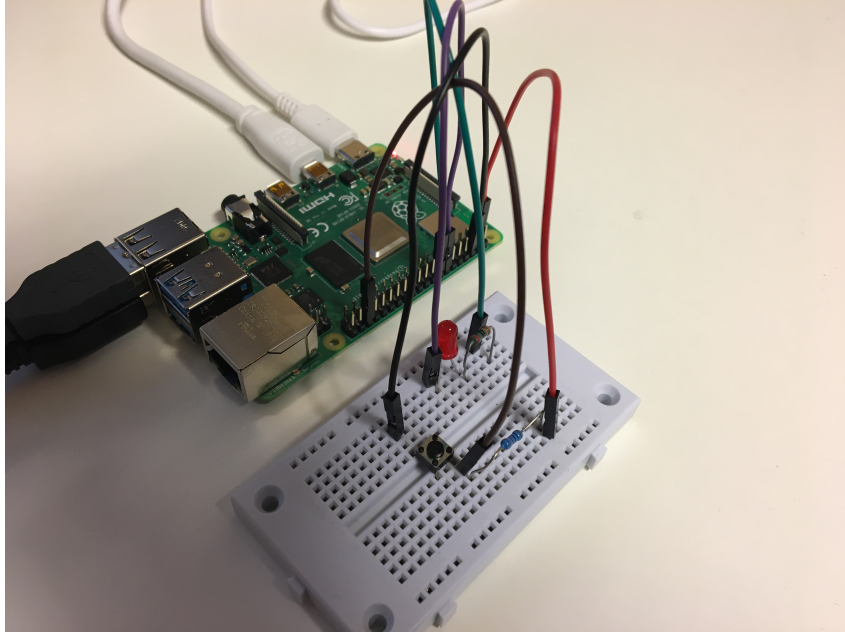


True/High



We Push the Button

Button + LED Example

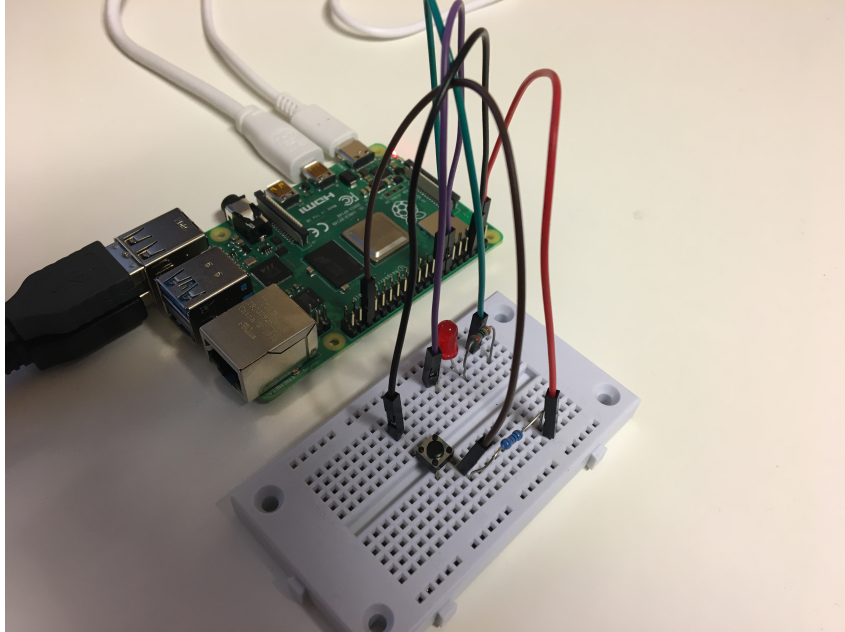


```
from gpiozero import LED, Button
from time import sleep
```

```
pin_btn = 16
button = Button(pin_btn)
pin_led = 23
led = LED(pin_led)
```

```
while True:
    if button.is_pressed:
        led.on()
    else:
        led.off()
    sleep(1)
```

Button + LED Example



```
import time
import RPi.GPIO as GPIO

# Pin definitions
led_pin = 23
btn_pin = 16

# Suppress warnings
GPIO.setwarnings(False)

# Use "GPIO" pin numbering
GPIO.setmode(GPIO.BCM)
# Set Button pin as input
GPIO.setup(btn_pin, GPIO.IN)
# Set LED pin as output
GPIO.setup(led_pin, GPIO.OUT)

# Blink forever
while True:
    if GPIO.input(btn_pin):
        GPIO.output(led_pin, GPIO.LOW) # Turn LED off
    else:
        GPIO.output(led_pin, GPIO.HIGH) # Turn LED on

    time.sleep(1)
```



SPI

Serial Peripheral Interface (SPI)

Hans-Petter Halvorsen

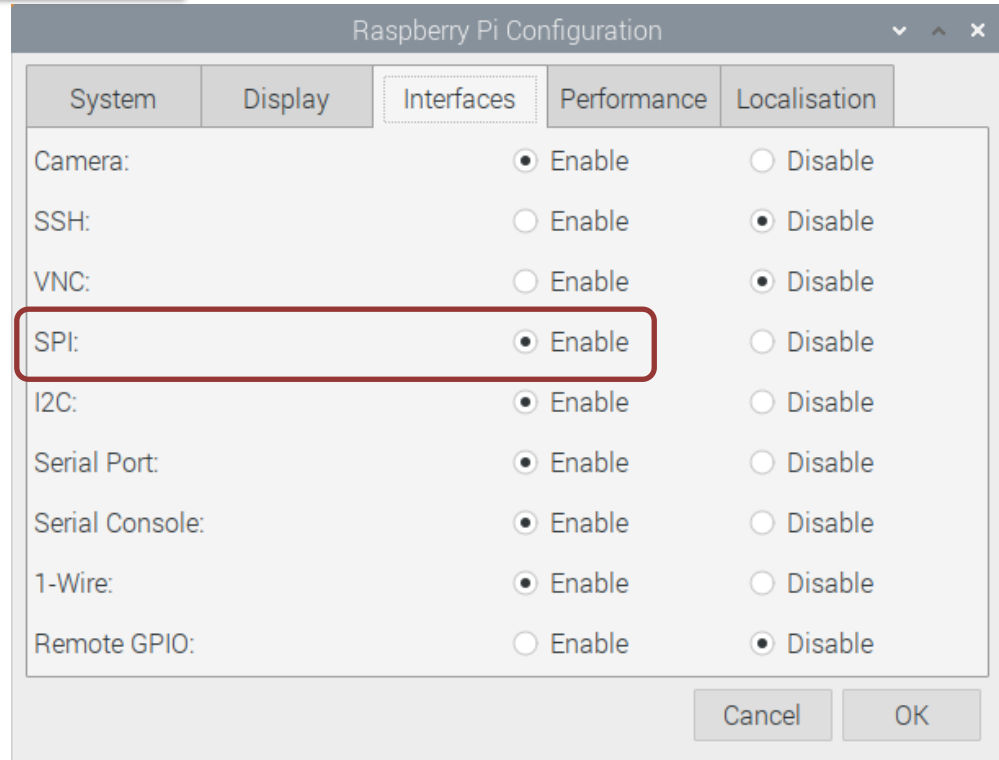
SPI

- Serial Peripheral Interface (SPI)
- SPI is an interface to communicate with different types of electronic components like Sensors, Analog to Digital Converts (ADC), etc. that supports the SPI interface
- Thousands of different Components and Sensors supports the SPI interface

<https://www.raspberrypi.org/documentation/hardware/raspberrypi/spi/>

Access SPI on Raspberry Pi

You need to Enable SPI on the Raspberry Pi



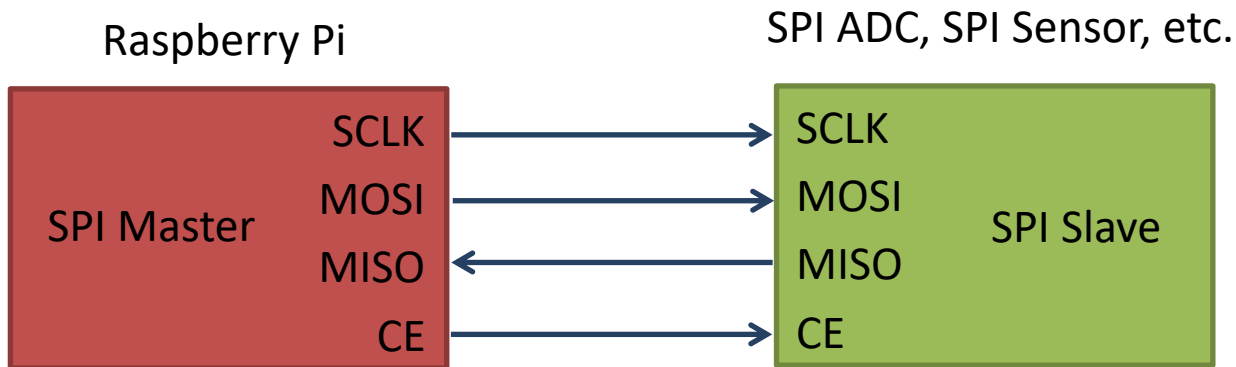
The screenshot shows the 'Raspberry Pi Configuration' window with the 'Interfaces' tab selected. The 'SPI' option is highlighted with a red rectangle, showing it is set to 'Enable'.

System	Display	Interfaces	Performance	Localisation
Camera:		☒ Enable		☐ Disable
SSH:		☐ Enable		☒ Disable
VNC:		☐ Enable		☒ Disable
SPI:		☒ Enable		☐ Disable
I2C:		☒ Enable		☐ Disable
Serial Port:		☒ Enable		☐ Disable
Serial Console:		☒ Enable		☐ Disable
1-Wire:		☒ Enable		☐ Disable
Remote GPIO:		☐ Enable		☒ Disable

Buttons: Cancel, OK

SPI Interface

SPI devices communicate in full duplex mode using a master-slave architecture with a single master

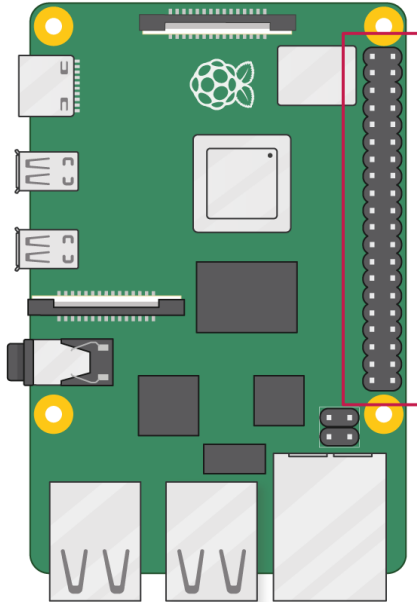


The SPI bus specifies four logic signals:

- **SCLK**: Serial Clock (output from master)
- **MOSI**: Master Out Slave In (data output from master)
- **MISO**: Master In Slave Out (data output from slave)
- **CE** (often also called SS - Slave Select): Chip Select (often active low, output from master)

SPI Wiring on Raspberry Pi

GPIO 40 pins Connector



3V3 power	1	2	5V power
GPIO 2 (SDA)	3	4	5V power
GPIO 3 (SCL)	5	6	Ground
GPIO 4 (GPCLK0)	7	8	GPIO 14 (TXD)
Ground	9	10	GPIO 15 (RXD)
GPIO 17	11	12	GPIO 18 (PCM_CLK)
GPIO 27	13	14	Ground
GPIO 22	15	16	GPIO 23
3V3 power	17	18	GPIO 24
GPIO 10 (MOSI)	19	20	Ground
GPIO 9 (MISO)	21	22	GPIO 25
GPIO 11 (SCLK)	23	24	GPIO 8 (CE0)
Ground	25	26	GPIO 7 (CE1)
GPIO 0 (ID_SD)	27	28	GPIO 1 (ID_SC)
GPIO 5	29	30	Ground
GPIO 6	31	32	GPIO 12 (PWM0)
GPIO 13 (PWM1)	33	34	Ground
GPIO 19 (PCM_FS)	35	36	GPIO 16
GPIO 26	37	38	GPIO 20 (PCM_DIN)
Ground	39	40	GPIO 21 (PCM_DOUT)



ADC

Analog to Digital Converter

Hans-Petter Halvorsen

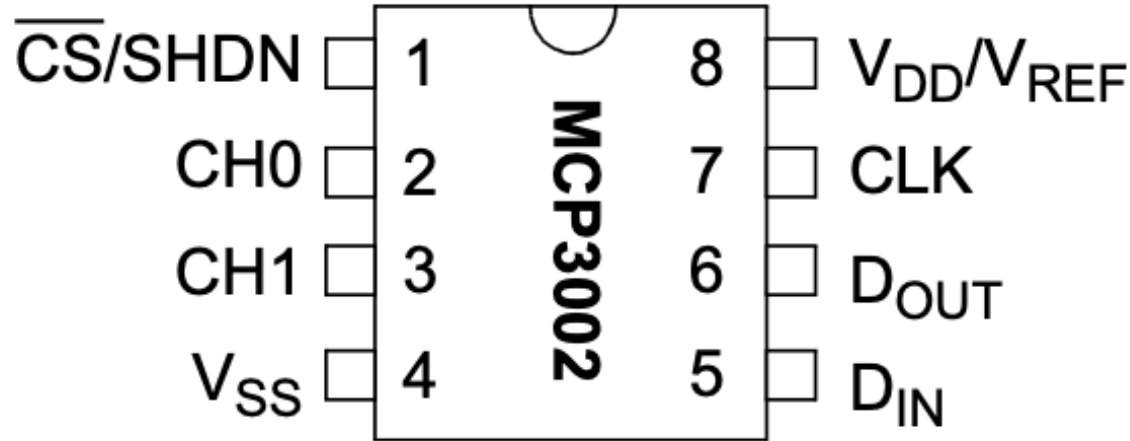
ADC

- The Raspberry Pi has only Digital pins on the GPIO connector
- If you want to use an Analog electric component or an Analog Sensor together with Raspberry Pi, you need to connect it through an external ADC chip
- ADC – Analog to Digital Converter

MCP3002 ADC chip

The MCP3002 is a 10-bit analog to digital converter with 2 channels (0-1).

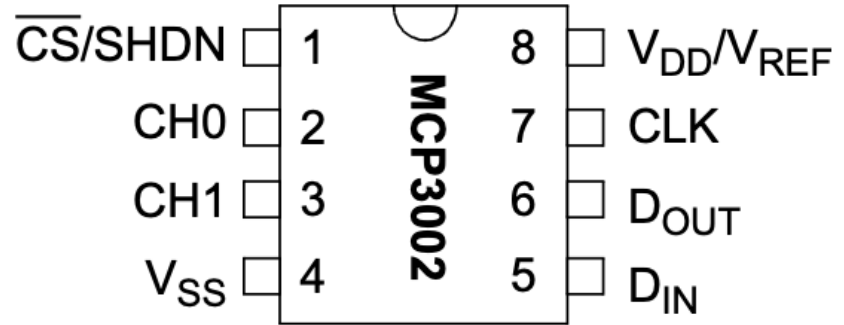
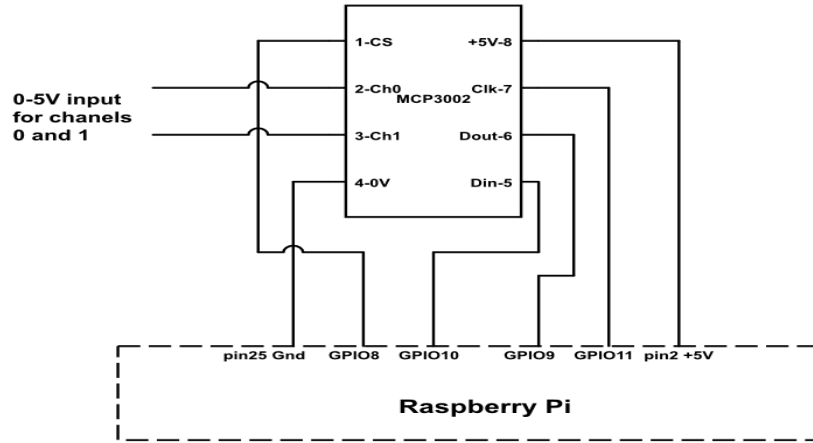
The MCP3002 uses a SPI Interface



<http://ww1.microchip.com/downloads/en/DeviceDoc/21294E.pdf>

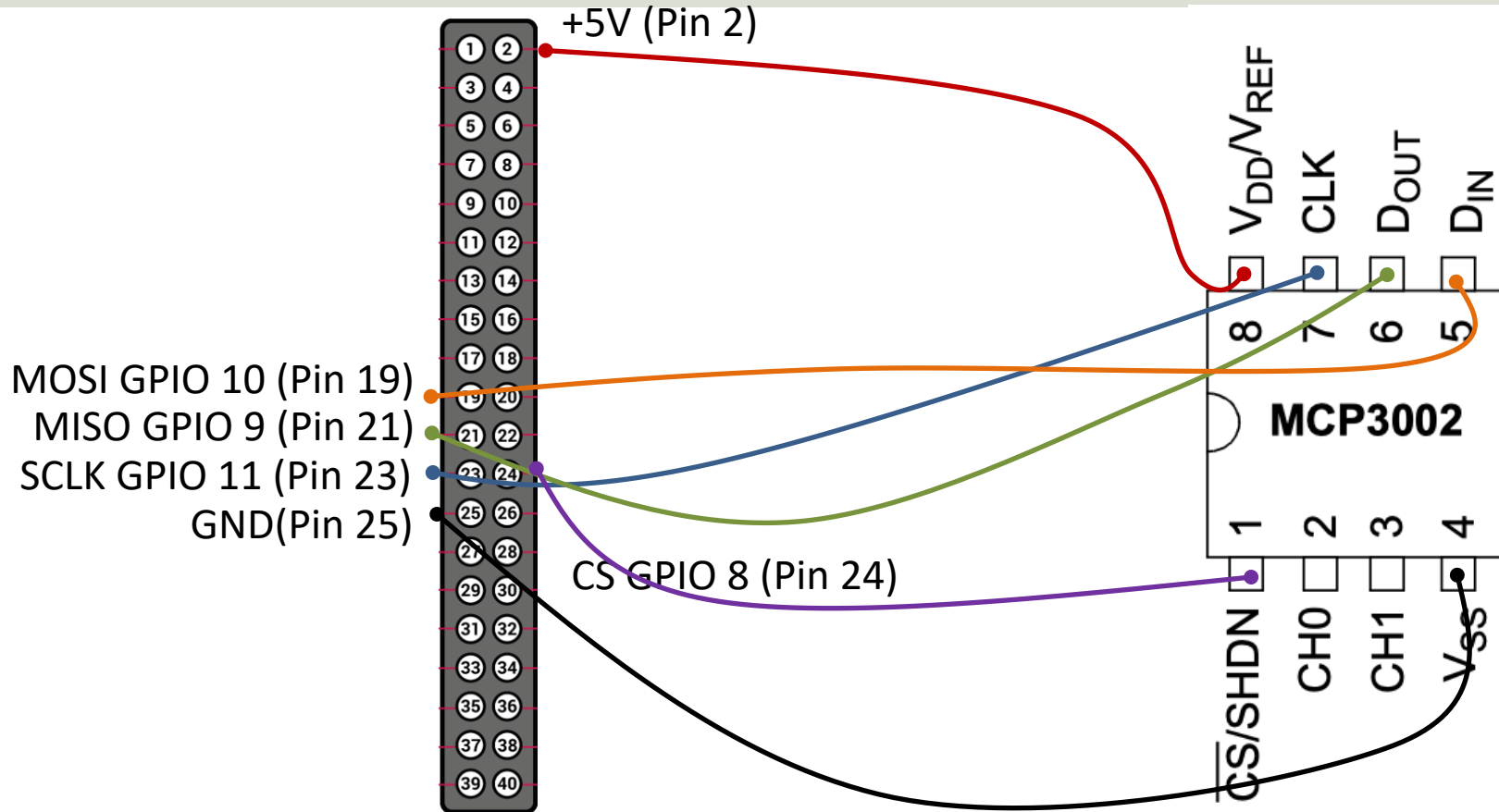
<https://learn.sparkfun.com/tutorials/python-programming-tutorial-getting-started-with-the-raspberry-pi/experiment-3-spi-and-analog-input>

Wiring



Wiring

Raspberry Pi GPIO Pins



GPIO Zero and MCP3002

```
gpiozero.MCP3002(channel=0, differential=False, max_voltage=3.3, **spi_args)
```

channel

The channel to read data from. The MCP3008/3208/3304 have 8 channels (0-7), while the MCP3004/3204/3302 have 4 channels (0-3), the MCP3002/3202 have 2 channels (0-1), and the MCP3001/3201/3301 only have 1 channel.

differential

If True, the device is operated in differential mode. In this mode one channel (specified by the channel attribute) is read relative to the value of a second channel (implied by the chip's design).

Please refer to the device data-sheet to determine which channel is used as the relative base value (for example, when using an MCP3008 in differential mode, channel 0 is read relative to channel 1).

value

The current value read from the device, scaled to a value between 0 and 1 (or -1 to +1 for certain devices operating in differential mode).

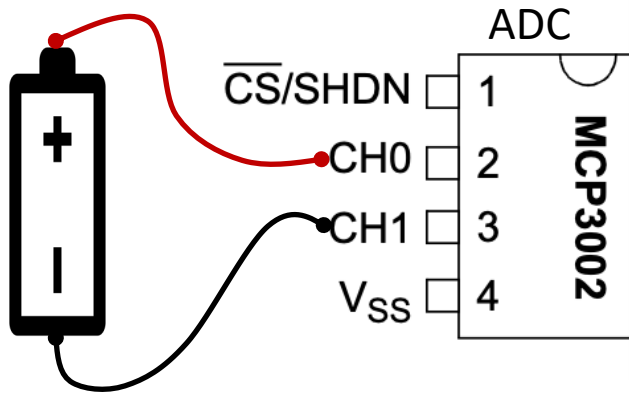
https://gpiozero.readthedocs.io/en/stable/api_spi.html

Read Data from ADC

For test purpose we start by wiring a 1.5V Battery to the CH0 (+) and CH1(-) pins on the ADC

Note! WE have set **differential=True** (meaning CH0 is “+” and CH1 is “-“)

1.5V Battery



```
from gpiozero import MCP3002
from time import sleep

adc = MCP3002(channel=0, differential=True)

N = 20

for x in range(N):
    adcddata = adc.value #Value between 0 and 1
    #print(adcddata)
    voltvalue = adcddata * 5 #Value between 0 and 5V
    print(voltvalue)
    sleep(1)
```

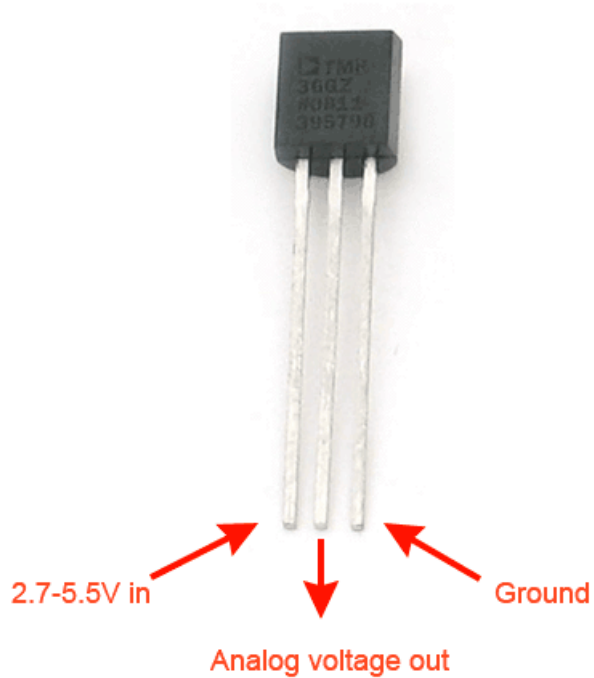


TMP36

Temperature Sensor

Hans-Petter Halvorsen

TMP36 Temperature Sensor

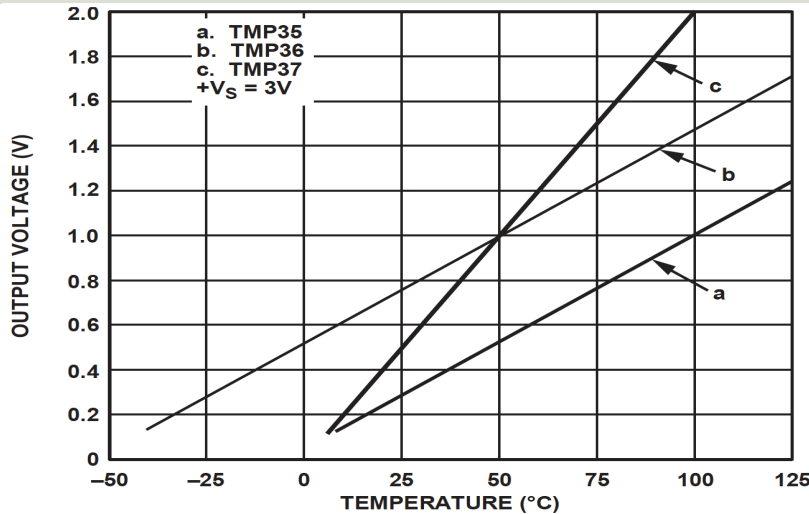


A Temperature sensor like TM36 use a solid-state technique to determine the temperature.

They use the fact as temperature increases, the voltage across a diode increases at a known rate.

<https://learn.adafruit.com/tmp36-temperature-sensor>

TMP36 Temperature Sensor



Convert from Voltage (V) to degrees Celsius

From the Datasheet we have:

$$(x_1, y_1) = (0.75V, 25^{\circ}C)$$

$$(x_2, y_2) = (1V, 50^{\circ}C)$$

There is a linear relationship between Voltage and degrees Celsius:

$$y = ax + b$$

This gives:

$$y - 25 = \frac{50 - 25}{1 - 0.75}(x - 0.75)$$

Then we get the following formula:

$$y = 100x - 50$$

We can find a and b using the following known formula:

$$y - y_1 = \frac{y_2 - y_1}{x_2 - x_1}(x - x_1)$$

Measure Temperature with an ADC

TMP36 Temperature Sensor



Wire a TMP36 temperature sensor to the first channel of an MCP3002 analog to digital converter and the other pins to +5V and GND

```
from gpiozero import MCP3002
from time import sleep

adc = MCP3002(channel=0, differential=False)

N = 10

for x in range(N):
    adcddata = adc.value #Value between 0 and 1
    #print(adcddata)

    voltvalue = adcddata * 5 #Value between 0V and 5V
    #print(voltvalue)

    tempC = 100*voltvalue-50 #Temperature in Celsius
    tempc = round(tempC,1)
    print(tempC)

    sleep(1)
```

<https://www.halvorsen.blog>



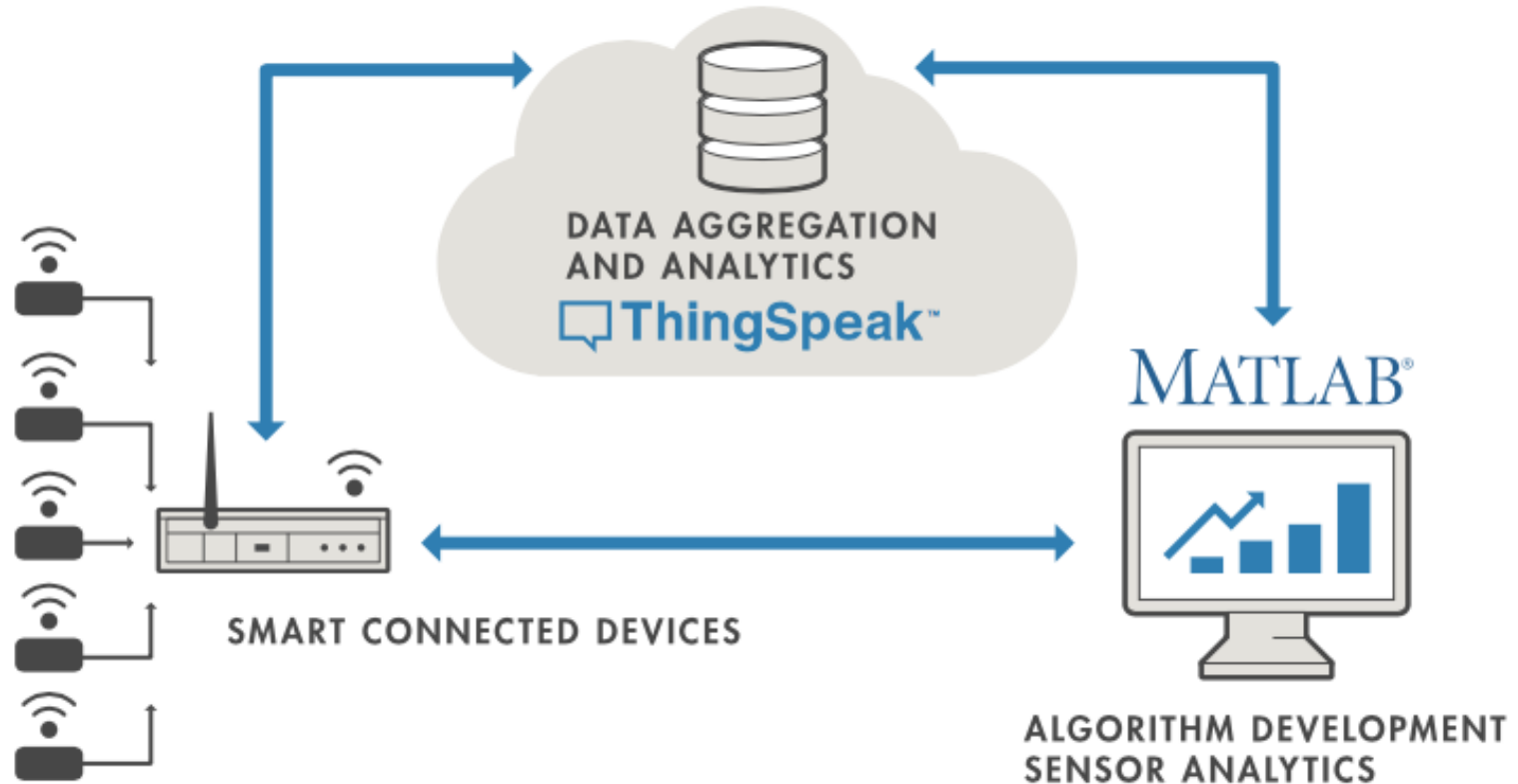
ThingSpeak

Hans-Petter Halvorsen

ThingSpeak

- ThingSpeak is an IoT analytics platform service that lets you collect and store sensor data in the cloud and develop Internet of Things applications.
- The ThingSpeak service also lets you perform online analysis and act on your data. Sensor data can be sent to ThingSpeak from any hardware that can communicate using a REST API
- ThingSpeak has a Web Service (**REST** API) that lets you collect and store sensor data in the cloud and develop Internet of Things applications (it also has **MQTT** API).
- <https://thingspeak.com>
- Python Library for ThingSpeak: <https://pypi.org/project/thingspeak/>

ThingSpeak



ThingSpeak Write

```
import thingspeak
import time

channel_id = xxxxxx
write_key  = "xxxxxxxxxxxxxxxxxxxx"

channel = thingspeak.Channel(id=channel_id, api_key=write_key)

N = 10
for x in range(N):
    temperature = 24
    response = channel.update({'field1': temperature})
    time.sleep(15)
```

<https://thingspeak.readthedocs.io/en/latest/api.html>

A Free ThingSpeak Channel can only be updated every 15 sec

Write TMP36 Data

```
import thingspeak
import time
from gpiozero import MCP3002
```

A Free ThingSpeak Channel can only be updated every 15 sec

```
adc = MCP3002(channel=0, differential=False)
```

```
channel_id = xxxxxxxx
write_key  = "xxxxxxxxxxxxxxxxxxxxxx"
```

```
channel = thingspeak.Channel(id=channel_id, api_key=write_key)
```

```
N = 10
```

```
for x in range(N):
```

```
    #Get Sensor Data
```

```
    adcddata = adc.value #Scaled Value between 0 and 1
```

```
    voltvalue = adcddata * 5 # Value between 0V and 5V
```

```
    tempC = 100*voltvalue-50 # Temperature in Celsius
```

```
    tempC = round(tempC,1)
```

```
    print(tempC)
```

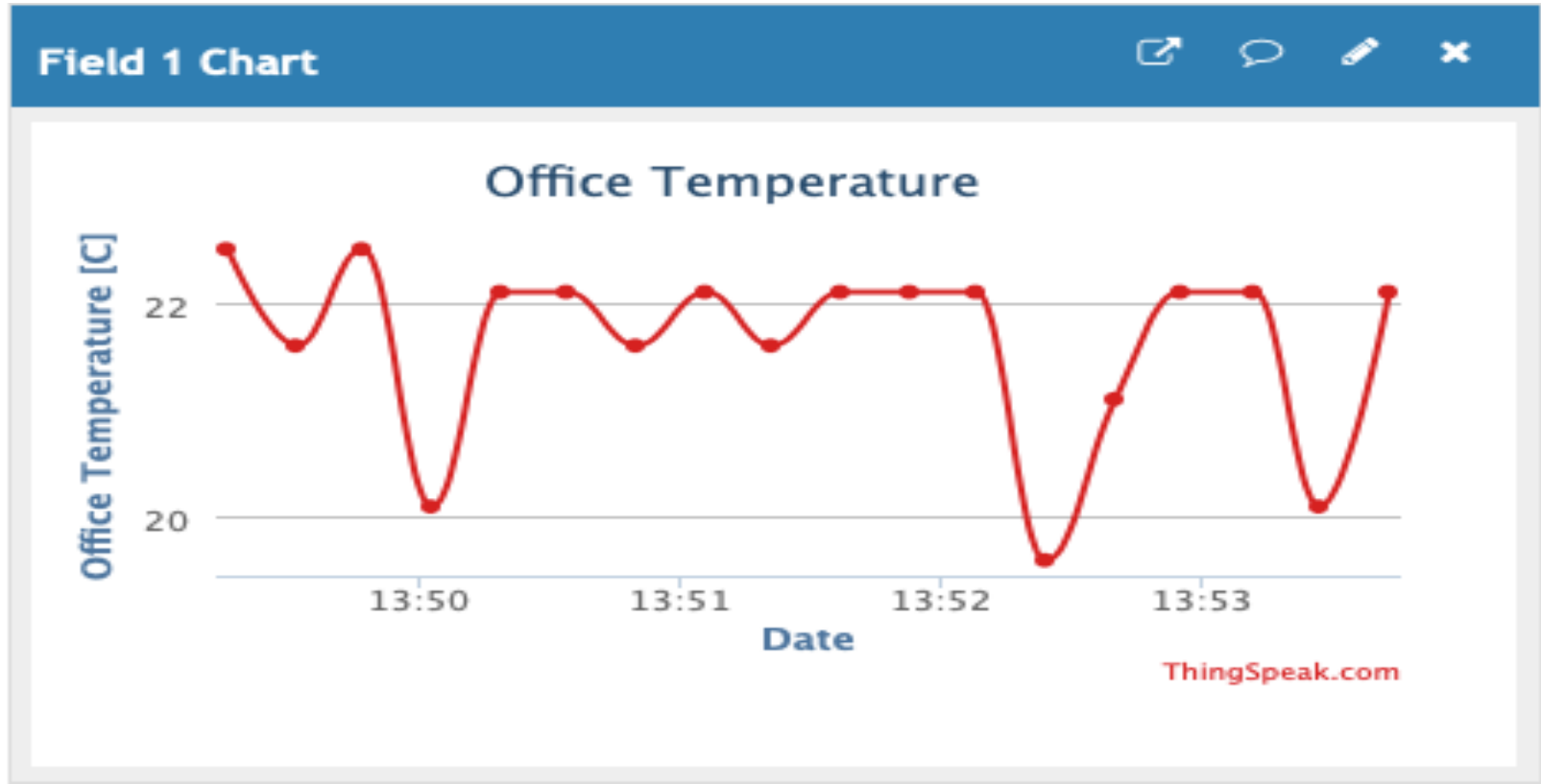
```
    #Write to ThingSpeak
```

```
    response = channel.update({'field1': tempC})
```

```
    time.sleep(15)
```


Write TMP36 Data

Here we see the Temperature Data in ThingSpeak:



ThingSpeak Read

```
import thingspeak

channel_id = xxxxxx
read_key   = "xxxxxxxxxxxxxxxxxxxx"

channel = thingspeak.Channel(id=channel_id, api_key=read_key)

#data = channel.get({})
data = channel.get_field({"field1"})

print(data)
```

<https://thingspeak.readthedocs.io/en/latest/api.html>